PagoGlobal

Infraestructura para una plataforma de pagos electrónicos de alcance global

Trabajo Práctico N.º 2 · Plataformas de Pagos Electrónicos

Matías Giachello · Octubre 2026

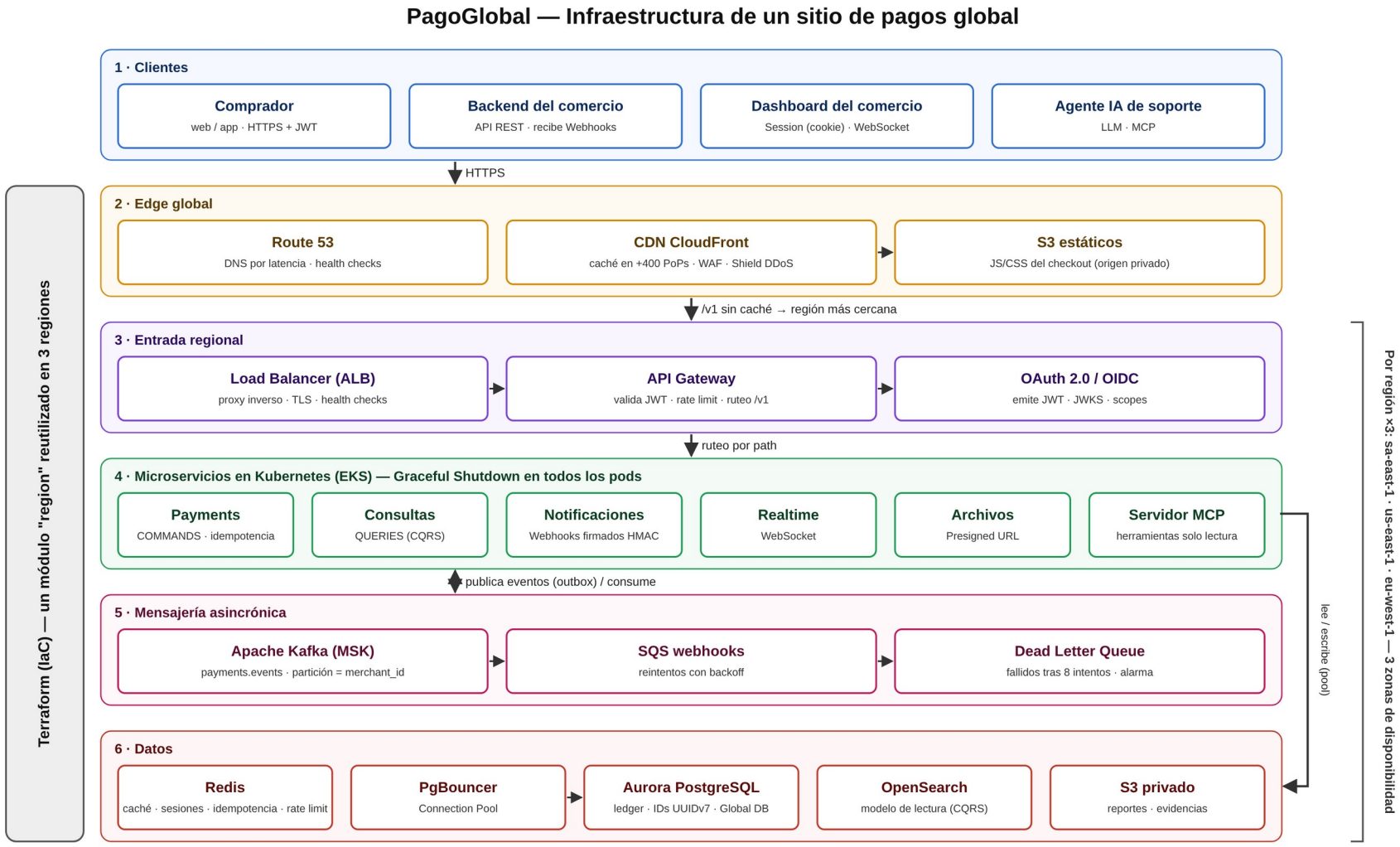
El caso: PagoGlobal

- Pasarela de pagos para comercios online: tarjetas, billeteras y transferencias
- Actores: comprador, comercio (API + dashboard), redes de tarjetas, agente de IA de soporte
- Tres regiones: San Pablo (LatAm), Virginia (Norteamérica), Irlanda (UE · GDPR)
- Funciones: cobrar, reembolsar, notificar al comercio, dashboard en vivo, reportes, login con Google, API para terceros
- Objetivo del TP: aplicar los 18 conceptos + Redis a un sitio global real

Requisitos no funcionales

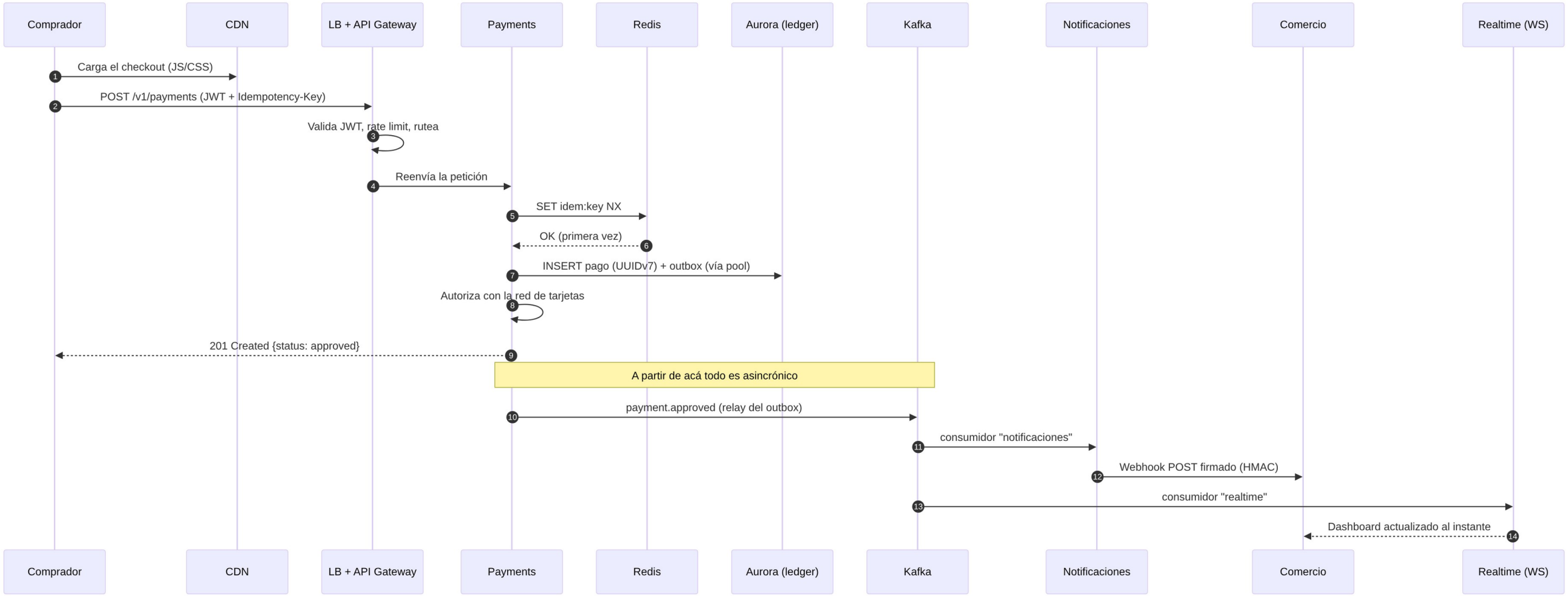
Atributo	Objetivo	¿Por qué?
Disponibilidad	99,99 % (≈ 52 min/año)	Estándar en pagos
Latencia	p99 < 300 ms	El comprador espera en el checkout
Throughput	2.000 TPS normal · 20.000 TPS pico	Black Friday / Hot Sale (×10)
Durabilidad	RPO = 0 en la región	No se puede perder un cobro
Recuperación	RTO ≤ 15 min ante caída de región	Conmutación a otra región
Idempotencia	Nunca cobrar dos veces	Reintentos por timeouts
Seguridad	PCI DSS · GDPR · TLS 1.2+	Regulación

Arquitectura por capas



Lo crítico es sincrónico (capas 1→2→3→4→6); el resto viaja por la capa 5. Aurora Global DB + MSK Replicator replican a la región secundaria (RTO ≤ 15 min).

Recorrido de un pago de punta a punta



Terraform — Infraestructura como Código

¿Qué es?

- Herramienta de HashiCorp; lenguaje declarativo HCL
- Describís QUÉ querés; terraform plan calcula el CÓMO
- Guarda un estado (state) de lo que ya existe
- Multi-nube: AWS, GCP, Azure…

En PagoGlobal

- Un módulo "region" escrito una vez, instanciado 3 veces
- VPC, EKS, Aurora, Redis, Kafka, SQS+DLQ, S3, ALB, API Gateway, CloudFront y Route 53
- Cambios por pull request + plan en CI (auditoría PCI)
- Black Friday: eks_min_nodes = 30 y apply
- State cifrado en S3 con lock en DynamoDB

⚖️ **Alternativa descartada: crear todo desde la consola → regiones distintas entre sí, sin auditoría y sin forma de reconstruir ante un desastre.**

CDN y Proxy inverso

¿Qué es?

- CDN: red de cachés distribuida que sirve estático cerca del usuario
- Proxy inverso: intermediario delante del backend; oculta servidores, termina TLS, balancea
- La CDN es, en sí, un proxy inverso global

En PagoGlobal

- CloudFront sirve el JS/CSS del checkout: ~20 ms vs ~250 ms al origen
- WAF + Shield en el edge: bloquea bots, inyecciones y DDoS
- Proxies inversos en cadena: CloudFront → ALB → Ingress → PgBouncer
- La API pasa por la CDN pero sin caché (no-store)
- Nada de la app tiene IP pública

⚖️ **La CDN solo cachea lo estático y público: las respuestas de pagos son privadas y dinámicas.**

Load Balancer vs API Gateway

	Load Balancer	API Gateway
Capa	L4 (TCP) o L7 básico	L7 con lógica de aplicación
Objetivo	Repartir entre instancias IGUALES	Rutear a servicios DISTINTOS
Entiende de	IPs, puertos, salud de instancias	Rutas, JWT, cuotas, versiones
Ejemplo	AWS ALB / NLB, HAProxy	AWS API Gateway, Kong
En PagoGlobal	ALB reparte entre réplicas del Gateway en 3 AZ	/payments → Payments · /reports → Consultas · rate limit por comercio

OAuth 2.0 y Session vs JWT

¿Qué es?

- OAuth 2.0: autorización delegada con scopes, sin compartir la contraseña
- Session: cookie con ID; el estado vive en el servidor (Redis) → revocable al instante
- JWT: token firmado con claims; se valida sin consultar nada → escala, pero no se revoca fácil

En PagoGlobal

- Authorization Code + PKCE: una plataforma e-commerce opera en nombre del comercio
- Client Credentials: ERP y servicio-a-servicio
- OIDC: "Iniciar sesión con Google"
- API → JWT de 10 min firmado ES256, validado en cualquier región con JWKS
- Dashboard → Session en Redis (HttpOnly, Secure, SameSite)

⚖️ **Usamos los dos: JWT donde importa escalar entre regiones; Session donde importa poder cerrar una cuenta robada en el acto.**

API, Webhook y MCP

¿Qué es?

- API: solicitud-respuesta; el cliente pregunta
- Webhook: "API inversa"; el servidor avisa con un POST cuando pasa algo
- MCP: protocolo abierto (Anthropic, 2024) para que un LLM use herramientas y datos de forma estandarizada

En PagoGlobal

- API REST /v1 para crear pagos y reembolsos
- Webhook payment.approved firmado con HMAC-SHA256 + timestamp (anti-replay)
- Entrega at-least-once → el comercio deduplica por event_id
- Servidor MCP con buscar_pago y motivo_rechazo: solo lectura, token con scope payments:read

⚖️ **Sin webhooks el comercio haría polling: miles de consultas inútiles. Al LLM nunca se le da una herramienta que mueva dinero.**

WebSocket vs Webhook

¿Qué es?

- WebSocket: conexión persistente, bidireccional, full-duplex sobre TCP
- Webhook: un POST por evento hacia el servidor de un tercero
- WS → a un navegador conectado ahora · Webhook → a un backend

En PagoGlobal

- WebSocket: el dashboard muestra cada venta al instante
- Webhook: el ERP del comercio marca el pedido como pagado
- Los dos salen del MISMO evento en Kafka (dos consumidores)
- Varios pods de Realtime: Redis Pub/Sub entre pods
- ALB con timeout largo para conexiones WS

⚖️ **Si el receptor está caído: el webhook se reintenta (y DLQ); el WS se pierde, por eso al reconectar el dashboard pide lo último por API.**

Kafka y arquitectura asincrónica con colas

¿Qué es?

- Kafka: log distribuido, particionado y replicado; los mensajes se retienen y se pueden releer
- Cola (SQS/RabbitMQ): tareas con reintento por mensaje
- Asincronía = desacople + tolerancia a fallos + absorber picos

En PagoGlobal

- Regla: lo que el comprador necesita para ver "aprobado" es sincrónico; el resto, asincrónico
- Tópico payments.events, partición = merchant_id (orden por comercio)
- 5 grupos de consumo: notificaciones, consultas, realtime, antifraude, contabilidad
- Outbox: pago y evento en la MISMA transacción
- acks=all, idempotencia del productor, 3 réplicas

⚖️ **Todo sincrónico sería más simple, pero si el servicio de emails se cae, se caerían los cobros.**

Dead Letter Queue

¿Qué es?

- Cola secundaria para mensajes que fallaron N veces
- Evita reintentos infinitos y que un mensaje "veneno" bloquee la cola
- Nada se pierde: queda para análisis y reproceso (redrive)

En PagoGlobal

- SQS webhooks con maxReceiveCount = 8 (~24 h con backoff)
- Error transitorio (timeout, 503) → reintento
- Error permanente (404, payload inválido) → directo a la DLQ
- Alarma si la DLQ tiene mensajes; el comercio puede reenviarlos desde el dashboard
- En Kafka: tópico payments.events.dlq

⚖️ **Sin DLQ: o descartás el mensaje (perdés la notificación de un pago) o reintentás para siempre.**

CQRS — separar escrituras de lecturas

¿Qué es?

- Command: cambia estado · Query: solo lee
- Cada lado con su modelo y su base optimizada
- Consecuencia: consistencia eventual en las lecturas

En PagoGlobal

- Escritura: Aurora PostgreSQL normalizada (ledger)
- Lectura: OpenSearch con documentos listos para el dashboard
- Se sincroniza consumiendo payments.events
- Reportes de 90 días no frenan a los pagos
- El estado puntual de un pago se lee del lado de comandos

⚖️ **Trade-off: un pago tarda ~1 s en aparecer en reportes. Aceptable para reportes, no para el estado del cobro.**

Redis — la memoria rápida del sistema

¿Qué es?

- Base clave-valor en RAM, latencia < 1 ms
- Estructuras: strings, hashes, listas, sets, sorted sets, streams
- TTL por clave; comandos atómicos (núcleo single-thread)
- Persistencia opcional (RDB / AOF)

En PagoGlobal

- Idempotencia: SET idem:{key} … NX EX 86400
- Rate limiting: INCR + EXPIRE por comercio y segundo
- Sesiones del dashboard (revocación con DEL)
- Caché cache-aside de configuración del comercio
- Pub/Sub entre pods de WebSocket

⚖️ **Redis NO es la fuente de verdad: si se cae, el sistema va a la base (más lento, pero sin perder pagos).**

Connection Pool

¿Qué es?

- Conjunto de conexiones abiertas y reutilizables
- Abrir una conexión a PostgreSQL = TCP + TLS + auth + proceso nuevo
- Préstamo y devolución; si se agota, se espera con timeout

En PagoGlobal

- Black Friday: 200 pods × 20 conexiones = 4.000 → PostgreSQL colapsa
- Pool local de 10 por pod + PgBouncer (transaction pooling)
- ≈ 300 conexiones reales a Aurora
- Timeout corto: si no hay conexión, 503 rápido
- Se cierra al final del Graceful Shutdown

⚖️ **Pool demasiado grande: satura la base. Demasiado chico: las peticiones esperan. Se mide y se ajusta.**

Secuencia vs UUID (y su relación con el Load Balancer)

	Secuencia	UUIDv4	UUIDv7 ✔️
¿Quién lo genera?	La base (central)	Cualquier instancia	Cualquier instancia
¿Adivinable?	Sí (IDOR)	No	No
Revela volumen	Sí	No	No
Índice B-tree	Excelente	Malo (aleatorio)	Muy bueno (ordenado por tiempo)
Multi-región	Conflictos	Sin conflictos	Sin conflictos
LB + IDs	Todas las instancias dependen de la base	Cada instancia genera sola	Ídem + Idempotency-Key compartida en Redis

Presigned URL

¿Qué es?

- URL de S3 con firma criptográfica y vencimiento
- Permite UNA acción sobre UN objeto durante poco tiempo
- Sin credenciales para el cliente y sin bucket público

En PagoGlobal

- Descarga del reporte de liquidación: URL válida 5 min
- Subida de evidencia de contracargo: POST prefirmado, máx. 10 MB, solo PDF
- El archivo va directo navegador ↔ S3: no consume CPU de los servicios
- Bucket privado, cifrado con KMS, auditado en CloudTrail

⚖️ **Alternativa descartada: pasar los archivos por la app (cara y lenta) o hacer público el bucket (inaceptable con datos de comercios).**

Graceful Shutdown

¿Qué es?

- Al recibir SIGTERM: dejar de aceptar → terminar lo que está en curso → cerrar recursos → salir
- Kubernetes lo envía en cada despliegue o al achicar el cluster
- terminationGracePeriodSeconds + preStop + readiness probe

En PagoGlobal

- /ready pasa a 503 → el ALB saca al pod de rotación
- Peticiones nuevas: 503 "reintente" (van a otro pod)
- El pago en curso termina y responde 201
- Commit de offsets de Kafka, cierre del pool, WS con código 1001
- Deregistration delay de 30 s en el target group

⚖️ **Sin esto: un pod muere entre la aprobación del banco y el INSERT → comprador cobrado sin registro.**

Estrategia global y recuperación ante desastres

- Route 53 por latencia: cada cliente va a la región sana más cercana
- Región de origen por comercio: sus pagos se escriben en un solo lugar (ledger consistente)
- Residencia de datos: comercios UE en eu-west-1, sin replicar fuera (GDPR)
- Cae un pod / nodo / zona → réplicas en 3 AZ, impacto de segundos
- Cae una región → Aurora Global DB (< 1 s de retraso) + MSK Replicator → RTO ≤ 15 min
- PCI DSS: tokenización del PAN en un vault aislado, HSM, mTLS, MFA, secretos rotados
- Simulacros trimestrales: un plan de DR no probado no existe

Prueba de fuego: Black Friday (×10)

Problema	Qué lo resuelve
Avalancha de visitas al checkout	CDN
Bots y abuso	WAF + rate limiting en el API Gateway con Redis
Reintentos por lentitud → cobro doble	Idempotency-Key en Redis
Miles de escrituras simultáneas	Connection Pool + PgBouncer
Comercios refrescando el dashboard	CQRS + WebSocket (en vez de polling)
Emails, webhooks, comisiones	Kafka y colas absorben el pico
Servidor de un comercio caído	Reintentos + DLQ
Autoscaling sube y baja pods	Graceful Shutdown
Pre-escalar la infraestructura	Terraform (eks_min_nodes = 30)

Conclusiones

- Ningún concepto alcanza solo: la robustez sale de combinarlos
- En pagos mandan la idempotencia y la consistencia: ledger transaccional, UUIDv7, outbox, consumidores idempotentes
- Separar lo sincrónico de lo asincrónico es la decisión de mayor impacto
- Cada decisión es un trade-off: CQRS → consistencia eventual; JWT → difícil revocar; UUID → más espacio
- Un sitio global no se construye a mano: Terraform hace las regiones idénticas y reconstruibles

¿Preguntas?

Código Terraform, ejemplos en Python y diagramas en la carpeta tp2-infraestructura-global